

Programming for Economist

PS4 +5, Git and DataProject
Class 9

Matias B.F. Hall,
Institute of Economics,
August 2026

KØBENHAVNS UNIVERSITET



Plan for today

1. Pandas
2. Git and Github
3. Data project/projects in general
4. Hints for PS4 + 5

1. Pandas - data work in Python

- Pandas is the standard tool for working with tabular data in economics
- With pandas we work with dataframe (df) and series objects. A df is a table and a series is a single column
- Typical workflow: load, clean, transform, merge, summarize and/or plot
- Core methods you will use a lot: `.loc[]`, `.groupby()`, `.merge()`, `.pivot_table()`
- Most errors come from wrong data types, missing values, or bad merge keys
- Check/inspect often using: `head()`, `tail()` `shape`, `dtypes`, and unique values in key columns
- Write code in small steps and keep only code you understand and can explain

2. Git and GitHub – version control

- Git tracks changes in your files over time
- GitHub stores your repositories online and makes sharing/collaboration easy
- In this course, code, notebooks, and updates are distributed through GitHub. Instead of downloading a new zip file every time there are changes, you can clone the repositories and Git will track changes, letting you update your local folders through VSCode.
- In VSCode, use Source Control to stage, commit, pull, push, and resolve conflicts
- Typical workflow: pull first, edit files, commit with clear message, push
- Commit often in small steps so you can roll back safely if something breaks

3. Projects in general

- Hand-in consists of one zip file containing everything
 - A single self-contained notebook (.ipynb) presenting the analysis
 - Documented Python module files (.py) (if any)
 - (optional) a README.md explaining what is in the hand in, dependencies and who is in the group
- Group creation:
 - All group members join one of the groups named 'Groups for projects #' found under 'People' in English or 'Grupper' in Danish.
 - Rename the group to your chosen name.
 - One of the group members hand in, and the hand in will automatically be registered as a group hand in.
- Structure both data and code
 - Notebook should run start to finish and give results
 - Don't repeat code too much, e.g. import once
 - Efficiency and speed - how many lines, what steps?
 - Difference between code used to solve the problem, and results

3. Projects in general

- Documentation
 - No need for a full docstring, but a little description about what the function does and maybe the inputs.
- No need to be perfect the first time
 - Focus on getting results first, then you can clean up your notebooks and .py files second.

Useful extensions:

- Default Python Kernels
 - (Maybe) Fixes the bug where you need to chose kernel every time
- Python extension pack
 - Couple of helpful tools
- Data Wrangler
 - Dataviewer useful when working in pandas/dataframes
- Jupyter TOC
 - Generate table of contents

4. Hints for PS4 + 5

- Run notebooks from top to bottom before interpreting results (many cells depend on earlier edits).
- (optional) Before drops/renames, make a safe copy: `df2 = df.copy()`.
- Use `df['column_name']` to select one column
- Use boolean filters with parentheses:
(`cond1`) & (`cond2`) and (`cond1`) | (`cond2`).
- Use `I = conditions` and `df.loc[I]` for filtering
- Use `df.loc[rows, 'column'] = value` to change selected observations
- After dropping rows, reset index when needed:
`df = df.reset_index(drop=True)`.
- Rename columns early so later merge/groupby/plot code stays simple:
`df = df.rename(columns={...})`.
- When combining categories into one series, use:
`groupby([...], as_index=False)['value'].sum()`.

4. Hints for PS4 + 5

- Before concatenating data from different tables, make sure:
 - column names match,
 - units are harmonized,
 - numeric columns are numeric (`astype(float)`).
- Use `df.drop(...)`, `df.rename(...)`, `df.sort_values(...)`, `df.reset_index(...)` for cleanup
- Use `pd.to_datetime(...)` before time operations
- When using DST/FRED: inspect metadata first, then download, then rename columns early.
- For DST use `tablesummary(...)` / `variable_levels(...)` before data pulls, so you request the right codes/data the first time.
- Start with PS4+5 part one -> DataProject
 - If you really love Pandas and working with data in Python, you can do part two on PS4+5 as the last thing:)